

CPR E 4890: Computer Networking and Data Communications

Lab Experiment #9 – CloudLab Experiment: TCP Congestion Control

(Total Points: 100)

Objective

Using CloudLab to evaluate a TCP congestion protocol by collecting, visualizing, and analyzing data.

Pre-Lab

Before the lab starts, you may follow the instructions below to prepare your CloudLab experiment.

Lab Expectations

Work through the lab and let the TA know if you have any questions. After the lab, write up a lab report and be sure to:

- Attend the lab. (5 points)
- Summarize what you learned in a few paragraphs. (15 points)
- Include your answers to all exercises with screenshots when asked. (80 points)

Procedure

Prepare Your Experiment

- 1) If you haven't already, log in CloudLab at: <https://www.cloudlab.us/login.php>
- 2) Select **Experiments** and **Start Experiment**.
- 3) Select **Change profile**, search for **cpre4890s26-tcpcongestion**, and select it.
- 4) Click **Select Profile** and then **Next**.
- 5) Use your net-id for the name of the experiment and select a cluster with a green dot next to it. Then select **Next** and then **Finish**.
- 6) Wait for your resources to be ready. This may take several minutes. If you receive an email regarding the image of your nodes being outdated, ignore it.
- 7) Log into **client** and **server** nodes via SSH. Use the same technique you used in Lab4.

Exercise 1: Generate Data

1. On the client, enable the tcp_probe tracepoint and set it to monitor traffic to/from TCP port 5001:

```
echo "sport == 5001 || dport == 5001" | sudo tee /sys/kernel/tracing/events/tcp/tcp_probe/filter
echo 1 | sudo tee /sys/kernel/tracing/events/tcp/tcp_probe/enable
```

2. On the **server**, run

```
iperf -s
```

3. On the **client**, run

```
sudo cat /sys/kernel/tracing/trace_pipe | tee /tmp/tcpprobe.dat
```

4. Open another SSH session to the **client** and run

```
iperf -t 60 -c server -P 3
```

5. Wait for iperf to complete on the client. It will take about a minute. In the window where trace_pipe is running, you should see a line of output for each TCP packet. Take screenshots of the trace and iperf output for the running results. (10 points)

6. After iperf completes, press Ctrl+C in the trace_pipe window to stop recording. The output will be saved in /tmp/tcpprobe.dat on the client. Use the following scp command from your local machine to transfer tcpprobe.dat for processing. Substitute your username or net ID for netid, the node's host name for hostname, and the SSH port number for port.

```
scp -i <private_key> -P <port> <netid>@<hostname>:<file_location> <dest_location>
```

Example:

```
scp -i <private_key> -P <port> <netid>@<hostname>:/tmp/tcpprobe.dat ./tcpprobe.dat
```

Note: The client port number may not be specified in Cloudlab. In such cases, the -P <port> option may be omitted.

The tcp_probe tracepoint records a line of output every time a packet is sent, if either the destination or source port number in the TCP packet header is 5001 (since we set the filter to port 5001). Each line of output includes the following fields in key=value format:

Field	Explanation
Time	Time (in seconds) since beginning of probe output
Source	Source address and port of the packet, as IP:port
Destination	Destination address and port of the packet, as IP:port
Bytes	Bytes in packet
Next	Next send sequence number, in hex format
Unacknowledged	Smallest sequence number of packets sent but unacknowledged, in hex format
cwnd	Size of send congestion window for this connection (in MSS)
ssthresh	Slow-start threshold for this connection (in MSS)
Send window	Send window size (in MSS) – set to the minimum of send CWND and receive window size
Smoothed RTT	Smoothed estimated RTT for this connection (in ms)
Receive window	Receiver window size (in MSS), received in the last ACK. This limit prevents the receiver buffer from overflowing, i.e., prevents the sender from sending at a rate that is faster than the receiver can process the data.

Note: The output format uses key=value pairs (e.g., snd_cwnd=10 ssthresh=2147483647). Each line also includes a timestamp, process name, and CPU number at the start. To extract a specific field for plotting, use grep or awk (e.g., grep -oP 'snd_cwnd=\K\d+' tcpprobe.dat).

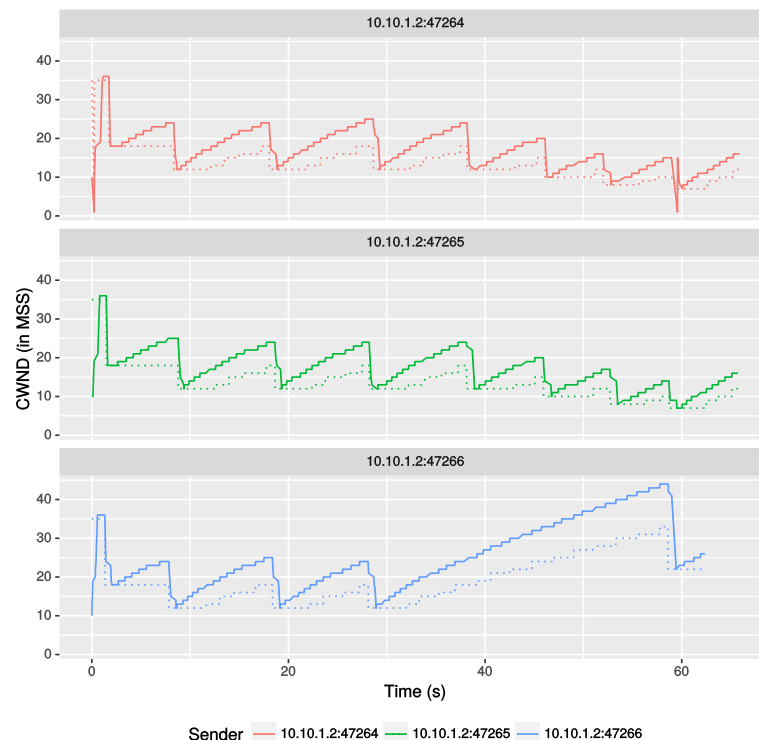
Exercise 2: Data Analysis

Create a plot showing the congestion window size and slow-start threshold for each TCP flow over the duration of the experiment, similar to Figure 1 below. **(15 points)**

There should be three graphs: one for each of the three iperf connections, indicated by the source port number (the IP Addresses will be the same).

Recommended plotting methods include converting .dat file to .csv and importing to either Excel or Google Sheets or using Python script using the Matplotlib library.

Figure 1 Example Plot of the TCP Congestion Window Size and Slow-Start Threshold



Annotate your plot to show the following: (20 points)

- Periods of **Slow Start**
- Periods of **Congestion Avoidance**
- Instances where 3 duplicate ACKs were received (which will trigger **Fast Recovery**)
- Instances of **Timeout**

Using your plot and/or experiment data, explain how the behavior of TCP is different in the **Slow Start** and **Congestion Avoidance** phases. (10 points)

In addition, using your plot, explain what happens to both the congestion window and the slow start threshold when 3 duplicate ACKs are received. (10 points)

Exercise 3: Clean Up Your Work

Terminate your experiments and take a screenshot of your empty experiment list after completing the lab. Include a screenshot in your lab report. (15 points)

Tips

- After 16 hours, your resources will automatically be terminated. If you wish to extend their reservation you must **Extend** the resources. This button is located next to **Terminate**.
- It is **recommended** to complete this lab on your own lab account so that your SSH keys will be on your student X drive. Using 489labuser will leave your keys on the lab computer unless extra steps are taken.
- Refer to Lab 4 on how to perform actions such as starting and terminating an experiment.
- Per RFC 5681, the initial value of ssthresh will be very high to begin. Make sure you manually set your y-axis limits to avoid these high values from dominating your plot and preventing the typical congestion control pattern in the steady state from being seen.